

SDP/PDS OS Internals

Date: 1/9/2025

Questions

Theoretical Part

Question 1

A) Consider the following statements regarding the possible advantages and disadvantages of an inverted page table (IPT) compared to a standard page table (PT). Tell each one whether it is true or false (YES/NO):

1. The IPT reduces the address space of processes.
2. If the size of a process' address space is equal to half the RAM size, the process' PT is generally smaller than the IPT.
3. If the size of a process' address space is equal to the RAM size (excluding the RAM allocated to the kernel), the process' PT and the IPT can have equal number of entries.
4. In the presence of multiple processes, each one with its own PT, an IPT would be a better alternative in terms of memory usage, because each process' address spaces is at least large as the RAM.
5. When the IPT is combined with a hash table, the size is doubled, because IPT entries are replicated in the hash table.
6. The hash table improves search time because the page number is the index in the hash array.

B) Consider a process P1, having a virtual addressing space of 72 GB, in a system equipped with 32GB of RAM, with 64-bit architecture (in which the Byte is addressed) and paged memory management (4KB pages/frames) for user processes only, so not for the kernel. Assume 8GB of RAM are statically allocated to the kernel. You want to compare a standard page table-based solution (one table for each process) and an IPT-based solution. Calculate:

- The number of entries in the Page Table.
- The number of entries in the IPT, assuming that the RAM allocated to the kernel is not mapped by the IPT.
- The minimum number of bits required for a frame number, assuming that the system could support RAM sizes up to 256GB.

- The minimum number of bits required for a page number, assuming that the system could support address spaces up to 4TB.
- The size of an IPT entry, assuming that a PID has to be represented on 12 bits, 6 bits are needed for flags, and the size is a power of 2.
- The ratio between the PT size and the IPT size, assuming that a PT entry needs to include 6 flag bits, and the size of the entry is a power of 2.

Question 2

A) A disk organized with RAID structure is given. Consider the mean time between failures (MTTF), the mean time to repair (MTTR), and the mean time between data loss (MTTDL).

- Can mirroring produce a MTTDL less than the MTTF (assume $MTTR \ll MTTF$)?
- Can striping reduce the MTTF of a given disk?

B) Consider a RAID structure with 4 disks in a "striped" configuration (RAID 0: non-redundant striping). Every disk has $MTTF=10000$ hours. Compute the MTTDL (considering a partial data loss, on one of the 4 disks).

C) Consider a RAID structure with 4 disks in a "striped+mirrored" configuration (striping on 2 disks, then mirroring with another 2 disks). If each of the disks can fail independently of the other, with $MTTF = 10000$ hours, compute the MTTR to ensure $MTTDL \geq 100 \times MTTF$?

Question 3

A) Consider disk scheduling algorithms. Answer the following questions (YES/NO):

- The C-SCAN algorithm optimizes positioning time for HDDs, by minimizing rotational latency.
- Given the same set of disk service requests, the total travelling distance of the disk head in C-SCAN is generally higher than in SCAN.
- NVM (Non-Volatile Memory) disks do not require the same scheduling algorithms used for HDDs.
- Given a disk request queue with N requests, no additional request can be accepted (and enqueued) while an STTF (Shortest Seek Time First) algorithm is servicing the N requests.
- Seek time minimization is the target of scheduling algorithm for HDDs.

B) A large text file contains N lines of fixed length (40 chars for each line), containing a set of sorted names. A binary (dichotomic) search algorithm is implemented directly on the file (either exploiting memory mapping of files or by properly using seek, read and write primitives). Depending on the underlying file system (file allocation strategy: contiguous, linked list, FAT or based on Inodes), we expect that (YES/NO):

- All allocation strategies will provide a logarithmic cost, as binary search is logarithmic and the algorithm and the allocation are independent each other, and operating at different SW layers.
- Contiguous allocation guarantees a logarithmic cost, as random/direct access is supported with constant time ($O(1)$).
- FAT and linked list do not support $O(1)$ random/direct access, so binary search becomes $O(N \log N)$: as a consequence linear search is better in term of complexity.
- Inodes do not support random/direct access, so they do not guarantee logarithmic binary search.
- Though FAT only support sequential access, it can be much more efficient than linked lists with binary search.

OS161

Question 4

Consider, in OS161, the implementation of virtual memory management. The `as_copy` function aims to "duplicate" an address space, i.e. to create a new address space, starting from an existing one, of which a copy is made. The function could be used, for example, to realize the system call `fork`, which generates a child process starting from a copy of an existing process (parent).

A possible implementation of `as_copy` is proposed:

```
1 int as_copy(struct addrspace *old, struct addrspace **ret) {
2     struct addrspace *new;
3
4     dumbvm_can_sleep();
5     new = as_create();
6
7     new->as_vbase1 = old->as_vbase1;
8     new->as_npages1 = old->as_npages1;
9     new->as_vbase2 = old->as_vbase2;
10    new->as_npages2 = old->as_npages2;
11
12    if (as_prepare_load(new)) {
13        as_destroy(new);
14        return ENOMEM;
15    }
16
17    memmove((void *) PADDR_TO_KVADDR(new->as_pbase1),
18            (const void *) PADDR_TO_KVADDR(old->as_pbase1),
19            old->as_npages1*PAGE_SIZE);
20
21    memmove((void *) PADDR_TO_KVADDR(new->as_pbase2),
22            (const void *) PADDR_TO_KVADDR(old->as_pbase2),
23            old->as_npages2*PAGE_SIZE);
24
25    memmove((void *) PADDR_TO_KVADDR(new->as_stackpbase),
26            (const void *) PADDR_TO_KVADDR(old->as_stackpbase),
27            DUMBVM_STACKPAGES*PAGE_SIZE);
28
29    *ret = new;
30    return 0;
31 }
```

Answer the following questions (or say YES/NO for true/false sentences):

- Is the function allocating physical RAM memory, when calling `as_prepare_load`?
- The explicit transformations from physical to logical address (e.g. `PADDR_TO_KVADDR(new->as_pbase1)`) could be avoided by simply using logical addresses (e.g. `new->as_vbase1`).
- If the `fork` system call is implemented using `as_copy`, the copy-on-write technique cannot be supported.

- The declaration of the parameter `ret`, using a double star, is an error: there should be a single star, and the last statement should be `ret = new`;
- The generated address space (pointed by `new`) is a user data structure, seen by a process when running in user mode.

Given the following values for `new->as_vbase1`, `new->as_pbase1`, `new->as_npages1`, `new->as_vbase2`, `new->as_pbase2`, `new->as_npages2`: 0x400000, 0x200000, 2, 0x480000, 0x300000, 4

- Which is the result of `PADDR_TO_KVADDR(new->as_pbase1)`?
- Which is the physical address for logical address 0x4000AB?
- Which is the kernel logical address for physical address 0x303000?

Question 5

Consider a user process on OS161. For each of the following sentences, say YES/NO (if they are true/false):

- The switchframe of a process is in its kernel stack and it is dynamically allocated (with `kmalloc`).
- The `switchframe_switch` function is implemented in assembly code: it switches context by saving the context of a thread and restoring the context of another thread.
- The user stack of a process is not part of its address space, which is limited to the code and data segments.
- In Dumbvm the user stack of a process has a fixed size of 18 pages.
- The `enter_new_process` function is used to start a user process by creating a trapframe and simulating a return from exception.
- The `enter_new_process` function waits for the process termination and returns the exit status of the user process.
- When a user process performs a `read(fd, buf, nb)`, `buf` is the source and `fd` the destination.
- When a user process performs a `read(fd, buf, nb)`, `buf` can be a pointer to a kernel buffer (if kernel buffering is enabled).
- The MIPS system call handler in OS161 (function `syscall`) receives all needed data in the trapframe: so the function has only one parameter, the pointer to a trapframe.
- When the `syscall` function terminates, it signals (function `V`) a semaphore on which the user process is waiting for the system call to terminate.

Answers

Theoretical Part

Question 1

A)

1. **NO.** The IPT reduces the memory overhead of the page table itself, not the address space of the process.
2. **YES.** Since generally the IPT size is proportional to the RAM size and the PT size is proportional to the virtual address space allocated, if the address space is half the RAM size, the PT will generally be smaller. Additionally, PT entries are generally smaller than IPT entries.
3. **YES.** If the size of a process' address space is equal to the RAM size, the process' PT and the IPT can have equal number of entries (assuming the entry sizes are the same, which is not always true).
4. **NO.** A process's address space can be much smaller than the physical RAM. For instance, if each process has allocated only a single page, individual PTs would be far less expensive in terms of memory than a full IPT.
5. **NO.** IPT entries are not replicated in the hash table. The hash table function simply guarantees that with a roughly constant cost, we can map to the correct IPT entry.
6. **NO.** The page number is not the index in the hash array; it is the search key that is computed through the hash function to find the corresponding entry.

B)

- **The number of entries in the Page Table:**

The process's virtual address space is 72 GB. The number of entries is the virtual address space divided by the page size:

$$72 \text{ GB} / 4 \text{ KB} = 18\text{M entries.}$$

- **The number of entries in the IPT, assuming that the RAM allocated to the kernel is not mapped by the IPT:**

The number of entries of the IPT is equal to the user RAM size divided by the frame size:

$$(32 \text{ GB} - 8 \text{ GB})/4 \text{ KB} = 6\text{M entries.}$$

- **The minimum number of bits required for a frame number, assuming that the system could support RAM sizes up to 256GB:**

$256 \text{ GB} = 2^8 \times 2^{30} \text{ Bytes} = 2^{38} \text{ Bytes}$, meaning a physical address requires 38 bits.

Subtracting the 12 bits needed for the 4KB offset ($2^{12} = 4096$):

$$38 - 12 = 26 \text{ bits for the frame number.}$$

- **The minimum number of bits required for a page number, assuming that the system could support address spaces up to 4TB:**

$4 \text{ TB} = 2^2 \times 2^{40} \text{ Bytes} = 2^{42} \text{ Bytes}$, meaning a logical address requires 42 bits.

Subtracting the 12 bits required by the offset:

$$42 - 12 = 30 \text{ bits for the page number.}$$

- **The size of an IPT entry, assuming that a PID has to be represented on 12 bits, 6 bits are needed for flags, and the size is a power of 2:**

The system requires 30 bits for the virtual page number, 12 for the PID, and 6 for flags.

$$30 + 12 + 6 = 48 \text{ bits.}$$

The nearest power of 2 greater than or equal to 48 is 64 bits (8 Bytes).

- **The ratio between the PT size and the IPT size, assuming that a PT entry needs to include 6 flag bits, and the size of the entry is a power of 2:**

Each PT entry needs 26 bits for the frame number and 6 bits for flags.

$26 + 6 = 32 \text{ bits}$. Since 32 is already a power of 2, the PT entry size is 32 bits (4 Bytes).

$$\text{Total PT size} = 18\text{M entries} \times 4 \text{ Bytes} = 72 \text{ MB.}$$

$$\text{Total IPT size} = 6\text{M entries} \times 8 \text{ Bytes} = 48 \text{ MB.}$$

$$\text{Ratio } (|PT|/|IPT|) = 72 \text{ MB}/48 \text{ MB} = 1.5.$$

Question 2 (RAID and Reliability)

A) Disks organized with RAID structure: MTTF, MTTR, MTDDL

- **Can mirroring produce a MTDDL less than the MTTF? (NO)**

With mirroring, data loss occurs only if both disks in a pair fail before the first one can be repaired. Assuming $\text{MTTR} \ll \text{MTTF}$, the probability of a second failure occurring during the short repair window is extremely small. Therefore, the system's MTDDL is exponentially larger than the MTTF of a single disk ($\text{MTDDL} \gg \text{MTTF}$).

- **Can striping reduce the MTTF of a given disk? (NO)**

The MTTF is an intrinsic hardware property of a physical disk. Striping (RAID 0) is a logical organization technique and does not alter the physical reliability or the physical MTTF of any *individual* disk. (Note: Striping *does* reduce the MTDDL of the overall array, but not the physical disk itself).

B) MTDDL for a 4-disk RAID 0 (Striped) configuration:

In a non-redundant RAID 0 configuration, the loss of any single disk results in a system data loss. Assuming the failures of the N disks are independent, the failure rate of the overall system (λ_{sys}) is the sum of the individual failure rates (λ).

$$\lambda_{sys} = N \times \lambda = 4 \times \frac{1}{MTTF}$$

The MTDDL of the system is the inverse of its failure rate:

$$MTDDL = \frac{1}{\lambda_{sys}} = \frac{MTTF}{4} = \frac{10000}{4} = \mathbf{2500 \text{ hours}}$$

C) MTTR for a 4-disk "Striped+Mirrored" configuration:

With 4 disks, the system is composed of 2 independent mirrored pairs.

1. The MTDDL for a *single* mirrored pair is:

$$MTDDL_{pair} = \frac{MTTF^2}{2 \times MTTR}$$

2. Since the entire system fails if *any* of the 2 mirrored pairs loses data, the system failure rate doubles, thereby halving the overall MTDDL:

$$MTDDL_{sys} = \frac{MTDDL_{pair}}{2} = \frac{MTTF^2}{4 \times MTTR}$$

Imposing the condition requested by the prompt ($MTDDL_{sys} = 100 \times MTTF$):

$$100 \times MTTF = \frac{MTTF^2}{4 \times MTTR}$$

$$400 \times MTTR \times MTTF = MTTF^2$$

$$400 \times MTTR = MTTF \implies MTTR = \frac{MTTF}{400}$$

Substituting the given value $MTTF = 10000$ hours:

$$MTTR = \frac{10000}{400} = \mathbf{25 \text{ hours}}$$

Question 3 (Disk Scheduling Algorithms)

A) True/False Statements on Disk Scheduling:

- The C-SCAN algorithm optimizes positioning time for HDDs, by minimizing rotational latency. (NO)

Motivation: Disk scheduling algorithms implemented by the Operating System (such as SSTF, SCAN, and C-SCAN) are designed to minimize **Seek Time** (the mechanical movement of the disk arm across cylinders). The OS generally lacks the real-time physical geometry data to optimize rotational latency; this task is typically deferred to the disk's internal hardware controller.

- **Given the same set of requests, the total travelling distance in C-SCAN is generally higher than in SCAN. (YES)**

Motivation: The SCAN algorithm acts like an elevator, servicing requests in both directions (sweeping back and forth). C-SCAN restricts scanning to a single direction; upon reaching the end of the disk, it performs a full, rapid seek back to the beginning *without servicing any requests* on the return trip. This "empty" return sweep significantly increases the total physical travelling distance of the head compared to SCAN.

- **NVM (Non-Volatile Memory) disks do not require the same scheduling algorithms used for HDDs. (YES)**

Motivation: NVM devices (such as Solid State Drives - SSDs) are entirely electronic and have absolutely no moving mechanical parts. Consequently, they do not suffer from Seek Time or Rotational Latency. Algorithms like SCAN or SSTF, which are explicitly designed to mitigate the cost of mechanical arm movement, are unnecessary and potentially detrimental to NVM performance. NVMs typically use simple FCFS or specialized flash-translation-layer scheduling.

- **No additional request can be accepted while an SSTF algorithm is servicing N requests. (NO)**

Motivation: The disk request queue is dynamic, meaning new I/O requests are continuously accepted and enqueued while the disk is active. In fact, this dynamic nature is the exact reason why the SSTF (Shortest Seek Time First) algorithm suffers from the **starvation** phenomenon: a continuous influx of new requests located near the current head position will cause the head to remain in that localized area, indefinitely delaying older requests located far away.

- **Seek time minimization is the target of scheduling algorithm for HDDs. (YES)**

Motivation: Seek time is the most significant bottleneck in HDD positioning time due to the slow mechanical movement of the disk arm. Since the OS maps logical block addresses sequentially to physical cylinders, it can easily calculate and minimize seek distances, making it the primary optimization target for OS-level disk scheduling.

B) Analysis of the statements (YES/NO):

- **All allocation strategies will provide a logarithmic cost, as binary search is logarithmic and the algorithm and the allocation are independent each other, and operating at different SW layers. (NO)**

Motivation: The logical independence of software layers does not imply performance independence. The final complexity of an algorithm executed on a disk is determined by the number of logical accesses required multiplied by the physical cost of resolving a single access (Random Access), which is dictated by the underlying File System.

- **Contiguous allocation guarantees a logarithmic cost, as random/direct access is supported with constant time ($O(1)$). (YES)**

Motivation: In contiguous allocation, locating the k -th block simply requires a basic mathematical addition (`base_address + offset`). Since the cost of each "jump" is $O(1)$, the overall complexity of the binary search remains unaffected: $O(\log N) \times O(1) = O(\log N)$.

- **FAT and linked list do not support $O(1)$ random/direct access, so binary search becomes $O(N \log N)$: as a consequence linear search is better in term of complexity. (YES)**

Motivation: Without $O(1)$ random access support, reaching a random offset requires sequentially traversing the linked pointers (an $O(N)$ cost). Executing $O(\log N)$ random jumps pushes the total cost to $O(N \log N)$. In contrast, a simple linear (sequential) search reads each block exactly once in order, yielding a total complexity of only $O(N)$.

- **Inodes do not support random/direct access, so they do not guarantee logarithmic binary search. (NO)**

Motivation: Inodes absolutely support pseudo-constant random access. Their asymmetric tree structure ensures that any block within the file can be reached by traversing a maximum of 3 or 4 levels of indirect indexing. Because the height of the tree is bounded by a constant, the access cost remains $O(1)$, thereby preserving the $O(\log N)$ complexity of the binary search.

- **Though FAT only support sequential access, it can be much more efficient than linked lists with binary search. (YES)**

Motivation: While both exhibit an $O(N)$ traversal complexity, their physical bottlenecks are completely different. In a standard Linked List, pointers are stored within the data blocks, requiring $O(N)$ slow *mechanical disk reads*. In a FAT system, all pointers are centralized in a table loaded entirely into *RAM*. The $O(N)$ traversal occurs at memory speeds, requiring the disk to perform only one final read to fetch the target block.

OS161

Question 4 (OS161 Virtual Memory and `as_copy`)

A) Analysis of the True/False Statements:

- **Is the function allocating physical RAM memory, when calling `as_prepare_load`? (YES)**

Motivation: In the baseline OS161 `dumbvm` implementation, `as_create` only allocates the kernel data structure (`struct addrspace`). It is exactly during `as_prepare_load` that the kernel calls `getppages()` to physically allocate the RAM frames required for the process segments.

- **Can explicit transformations like `PADDR_TO_KVADDR` be avoided by simply using logical addresses like `new->as_vbase1`? (NO)**

Motivation: The address `new->as_vbase1` is a *user-space* virtual address. The kernel cannot safely use `memmove` on user virtual addresses, especially when constructing an address space that is not currently active in the MMU/TLB. Instead, the kernel must use its own direct-mapped virtual segment (in MIPS, this is `kseg0`). The `PADDR_TO_KVADDR` macro converts the physical address into a safe `kseg0` kernel address so the kernel can write the copied data directly into physical RAM.

- **If fork is implemented using `as_copy`, the copy-on-write technique cannot be supported. (YES)**

Motivation: The `as_copy` function is designed to perform a *deep copy*: it immediately allocates new physical memory frames and physically copies all data bytes from the parent to the child. Copy-on-Write (CoW) fundamentally requires sharing the same physical frames (marked as read-only) and delaying the copy. A standard `as_copy` forces immediate duplication, thereby defeating the CoW mechanism.

- **The declaration of the parameter `ret`, using a double star, is an error. (NO)**

Motivation: In C, to modify a pointer belonging to the caller function so that it points to the newly allocated `struct addrspace`, the pointer itself must be passed by reference. Therefore, a pointer-to-pointer (`**ret`) is strictly required, and the assignment `*ret = new;` is absolutely correct.

- **The generated address space (pointed by `new`) is a user data structure, seen by a process when running in user mode. (NO)**

Motivation: `struct addrspace` is an internal kernel data structure. User processes have no knowledge of it and cannot access it due to hardware memory protection; it resides entirely in kernel memory space.

B) OS161 / MIPS Memory Address Calculations:

Given values:

`vbase1 = 0x400000`, `pbase1 = 0x200000`, `npages1 = 2`

`vbase2 = 0x480000`, `pbase2 = 0x300000`, `npages2 = 4`

1. Result of `PADDR_TO_KVADDR(new->as_pbase1)`:

In MIPS OS161, the kernel virtual memory segment (`kseg0`) maps directly to physical memory starting at base address `0x80000000`. The macro simply adds this base to the physical address:

$$0x200000 + 0x80000000 = \mathbf{0x80200000}$$

2. Physical address for logical address `0x4000AB`:

The logical address belongs to segment 1 (base `0x400000`). The offset is:

$$0x4000AB - 0x400000 = 0xAB$$

Adding this offset to the physical base of segment 1 (`pbase1`):

$$0x200000 + 0xAB = \mathbf{0x2000AB}$$

3. **Kernel logical address for physical address 0x303000:**

Using the same MIPS `kseg0` translation rule as in point 1, we add the `0x80000000` kernel base to the physical address:

$$0x303000 + 0x80000000 = \mathbf{0x80303000}$$

Question 5 (OS161 Processes and System Calls)

Analysis of the True/False Statements (YES/NO):

- **The switchframe of a process is in its kernel stack and it is dynamically allocated (with `kmalloc`). (NO)**
Motivation: While the `switchframe` is indeed located in the kernel stack, it is *not* dynamically allocated on the heap using `kmalloc`. It is allocated directly on the stack by decrementing the stack pointer (`sp`) during the assembly-level context switch.
- **The switchframe switch function is implemented in assembly code: it switches context by saving the context of a thread and restoring the context of another thread. (YES)**
Motivation: Context switching requires direct manipulation of CPU registers (like the program counter, stack pointer, and callee-saved registers) which cannot be done in standard C. It is implemented in assembly (e.g., `switch.S`).
- **The user stack of a process is not part of its address space, which is limited to the code and data segments. (NO)**
Motivation: The user stack is a fundamental component of the process's logical address space, typically located at the highest accessible virtual addresses and growing downwards.
- **In Dumbvm the user stack of a process has a fixed size of 18 pages. (YES)**
Motivation: In the baseline OS161 dumbvm implementation, the user stack size is hard-coded by the constant `DUMBVM_STACKPAGES`, which is defined as 18.
- **The `enter_new_process` function is used to start a user process by creating a trapframe and simulating a return from exception. (YES)**
Motivation: This function crafts a fake `trapframe` on the kernel stack and pushes it to the `mips_usermode` routine, which executes a return-from-exception (`rfe`) instruction to jump the CPU into user mode and begin executing the program.
- **The `enter_new_process` function waits for the process termination and returns the exit status of the user process. (NO)**
Motivation: This function *never returns* to the caller. It permanently transfers execution to user mode. Waiting for termination and retrieving the exit status is the job of the `sys_waitpid` system call.
- **When a user process performs a `read(fd, buf, nb)`, `buf` is the source and `fd` the destination. (NO)**

Motivation: The semantics of `read` are to extract data *from* a file and place it *into* memory. Therefore, `fd` (the file descriptor) is the source, and `buf` (the memory buffer) is the destination.

- **When a user process performs a `read(fd, buf, nb)`, `buf` can be a pointer to a kernel buffer. (NO)**

Motivation: `buf` is a pointer provided by the user process and must strictly point to a valid address within the user's address space. If a user were allowed to pass a kernel address, it would constitute a severe security vulnerability, allowing the user to arbitrarily overwrite kernel memory.

- **The MIPS system call handler (function `syscall`) receives all needed data in the trapframe: so the function has only one parameter, the pointer to a trapframe. (YES)**

Motivation: The assembly exception handler captures the entire state of the user registers (including the system call code in `v0` and arguments in `a0-a3`) into a `trapframe` structure on the kernel stack, and passes solely the pointer to this structure to the C `syscall()` dispatcher.

- **When the `syscall` function terminates, it signals (function `V`) a semaphore on which the user process is waiting for the system call to terminate. (NO)**

Motivation: A system call runs synchronously within the *same thread* as the user process (just in kernel mode). It does not spawn a concurrent thread that requires semaphore synchronization. When the `syscall` function completes, it simply returns to the assembly exception handler, which restores the registers and seamlessly resumes user mode execution.